P2413R1
Remove unsafe conversions of unique_ptr<T>

Published Proposal, 2024-05-22

Author:
Lénárd Szolnoki

Audience:
LEWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Table of Contents

§ 1. Revision history

§ 1.1. R1

— Allow conversions where the target type has destroying `operator delete`.

— Constrain `default_delete::operator()`.

— Constrain member functions of `unique_ptr` that take raw pointer arguments.

— Handle incomplete types.

— Document breaking changes.

§ 1.2. R0

— Constrain the conversion operator of `default_delete`.

§ 2. Motivation

Smart pointers are a success story of modern C++ as they mitigate many dangers of manual memory management. While smart pointers do provide "misusable" named functions (`reset`, `release`), their value-semantic operations (such as assignment and implicit conversions) are generally "easy to use, *impossible* to misuse."

But there is one gap in `unique_ptr`'s safety: sometimes it permits implicit conversions that are actually unsafe. Consider the following program:

```
#include <memory>

struct Base {};
struct Derived : Base {};

int main() {
    std::unique_ptr<Base> base_ptr = std::make_unique<Derived>();
}
```

The delete expression evaluated in the destructor of `base_ptr` deletes an object with dynamic type `Derived` and static type `Base`. As `Base` does not have a virtual destructor the behavior is undefined.

The goal of this proposal is to make this and other similar erroneous constructions of a `unique_ptr` to a base type ill-formed. As a result base classes with public non-virtual destructors become safer to use.

§ 3. Problematic constructs

§ 3.1. Problematic constructions of `unique_ptr<Base>`

All of the following operations eventually result in undefined behavior, unless the pointer gets released from the resulting `unique_ptr` before its destruction:

```
struct NonPolyBase {};
struct NonPolyDerived : NonPolyBase {};

// (1) conversion between unique_ptr types
std::unique_ptr<NonPolyBase> ptr1 = std::make_unique<NonPolyDerived>();

// (2) raw pointer construction
std::unique_ptr<NonPolyBase> ptr2(new NonPolyDerived{});
std::unique_ptr<NonPolyBase> ptr3(new NonPolyDerived{}, std::default_delete

// (3) reset(raw_ptr)
void f(std::unique_ptr<NonPolyBase>& uptr) {
  uptr.reset(new NonPolyDerived{});
}
```

§ 3.2. ODR-violation involving `unique_ptr` conversion and incomplete class types

`unique_ptr` and `default_delete` are among the few class templates in the standard library that support incomplete types for their first template parameter. Currently the convertiblity between specializations of these class templates is constrained on whether the corresponding raw pointer types are convertible, which is typically implemented with `std::is_convertible`. As validity of the conversion between the pointer types can change when the types are completed this can result in ODR-violation.

```
struct Base {
    virtual ~Base();
};
struct Derived;

struct S {
    S(std::unique_ptr<Derived>&&) {}
```

```
};

void foo(int, std::unique_ptr<Base>); // 1
void foo(long, S); // 2

void bar(std::unique_ptr<Derived>&& ptr) {
    foo(1, std::move(ptr)); // calls 2, ptr is not convertible to std::unic
    // in the overload resolution std::is_convertible<Derived*, Base*> gets
    // with value "false"
}

struct Derived : Base {};

void baz(std::unique_ptr<Derived>&& ptr) {
    foo(1, std::move(ptr)); // is_convertible<Derived*, Base*> would change
}
```

If we can assume that `is_convertible<Derived*, Base*>` is used for the constraints, then the program has undefined behavior according to [meta.rqmts]/5:

> 5 Unless otherwise specified, an incomplete type may be used to instantiate a template specified in [type.traits]. The behavior of a program is undefined if:
>
> (5.1) — an instantiation of a template specified in [type.traits] directly or indirectly depends on an incompletely-defined object type T, and
>
> (5.2) — that instantiation could yield a different result were T hypothetically completed.

This proposal makes the implicit conversion sequence from `std::move(ptr)` to `std::unique_ptr<Base>` a hard error (ill-formed outside of immediate context) when `Derived` is still incomplete, therefore the first call to `foo` in the code above will be ill-formed with diagnostics required.

§ 4. Proposed resolution

The resolution avoids undefined behavior at the destruction of `unique_ptr` due to an undefined delete expression as specified in [expr.delete]/3:

> 3 In a single-object delete expression, if the static type of the object to be deleted is not similar ([conv.qual]) to its dynamic type and the selected deallocation function (see below) is not a destroying operator delete, the static type shall be a base class of the dynamic type of the object to be deleted and the static type shall have a virtual destructor or the behavior is undefined.

This requires the ability to check two traits for the target type:

— `has_virtual_destructor_v<T>`, which checks whether the type has a virtual destructor declared.

— *`has-destroying-delete`*`<T>`, which is satisfied when the expression `delete p` for a `p` of type `T*` would select a destroying operator delete for the deallocation function. *`has-destroying-delete`* is used as an exposition-only concept throughout the proposal.

Whether the resulting delete expression would be otherwise undefined due to operations within the destructor or selected deallocation function is out of scope of this proposal.

§ 4.1. Design principles

— Avoid hard-coding `default_delete` into preconditions of `unique_ptr` member functions.

  — Custom deleters should be able to constrain `unique_ptr` conversions through a similar mechanism, if they wish to do so.

— Prefer SFINAE/concept friendliness.

  — Type traits and concepts shouldn't lie, if avoidable. `is_convertible_v<unique_ptr<T>, unique_ptr<U>>` might be `true` or `false`. This matches the current design as well.

— Avoid ODR-issues related to incomplete types.

  — `is_convertible_v<unique_ptr<T>, unique_ptr<U>>` might be unknown before `T` and `U` are completed. This should be a hard error.

The goal of the proposal with these high level principles can be met with the following approach:

— Constrain both the converting constructor and `operator()` member of single-object `default_delete` the same way to disallow converting to a base class without a virtual destructor or destroying operator delete.

— This is similar to the design of array `default_delete`, where only qualification conversions are allowed for both members.

— Constrain the raw pointer taking member functions of `unique_ptr` to only accept pointers that can be deleted by the target deleter's `operator()` directly.

— This essentially makes `operator()` of a deleter a customization point to control whether the raw pointer operations of `unique_ptr` should be allowed for certain arguments.

§ 4.2. *safely-convertible-for-delete* helper concept

This proposal introduces the following exposition-only helper concept:

```
template <typename T, typename U>
concept safely-convertible-for-delete = /*see below*/
```

safely-convertible-for-delete`<T, U>` is `true` for complete types `T` and `U` if and only if for some function `T create();` the following code at block scope is well-formed and has defined behavior:

```
{
  U* ptr = new T(create());
  delete ptr;
}
```

otherwise it is `false`.

If either of `T` and `U` is incomplete then the concept evaluates to `true` if for any possible completion of the types the concept evaluates to `true`. It evaluates to `false`, if for any possible completed types the concept evaluates to `false`. Otherwise *safely-convertible-for-delete*`<T, U>` is ill-formed, outside of an immediate context.

The most complex case is when `T` and `U` are distinct class types, where either or both of them are incomplete. Some notable examples:

— If only `T` is complete, then it can't be possibly derived from `U`, therefore the conversion from `T*` to `U*` is ill-formed, no matter how `U` is completed.

— If only `U` is complete and it's a `final` class then `T` can't be completed to derive from `U`, therefore the conversion from `T*` to `U*` is ill-formed, no matter how `T` is completed.

— If only `U` is complete and it doesn't have a virtual destructor or a destroying operator delete then `delete ptr;` above have undefined behavior, no matter how `T` is completed.

A possible implementation of this concept is in the appendix.

§ 4.3. Changes to `default_delete` and `unique_ptr`

Apply the following constraint for single-object `default_delete<T>`'s converting constructor and `operator()`:

```
template <typename U>
requires safely-convertible-for-delete<U, T>
constexpr default_delete(const default_delete<U>& d) noexcept;

template <typename U>
requires safely-convertible-for-delete<U, T>
constexpr void operator()(U* ptr) const;
```

Replace the raw-pointer taking member functions of single-object `unique_ptr` with member function templates and apply the following constraints:

```
template <typename U>
requires requires(deleter_type d, U p){ d(p); }
         && is_convertible_v<U&, pointer>
explicit constepxr unique_ptr(U p) noexcept;

template <typename U>
requires requires(deleter_type d, U p){ d(p); }
         && is_convertible_v<U&, pointer>
constexpr unique_ptr(U p, see below d1) noexcept;

template <typename U>
requires requires(deleter_type d, U p){ d(p); }
         && is_convertible_v<U&, pointer>
constexpr unique_ptr(U p, see below d2) noexcept;

template <typename U = pointer>
requires requires(deleter_type d, U p){ d(p); }
         && is_convertible_v<U&, pointer>
constexpr void reset(U p = pointer());
```

Some additional care need to be taken, so passing `nullptr` remains working.

Constrain further the raw-pointer taking member functions of array `unique_ptr`:

```
template <typename U>
requires (is_same_v<U, pointer>
          || (is_same_v<pointer, element_type*>
              && is_pointer_v<U>
              && is_convertible_v<U const*, pointer const*>
             )
         ) && is_invocable_v<deleter_type&, U&>
constexpr explicit unique_ptr(U p) noexcept;

// Similarly to the rest of the pointer-taking member functions
/*...*/
```

This is mainly for consistency with the single-object `unique_ptr` changes. This causes no material changes to `unique_ptr<T[], default_delete<T[]>>`, but could be used by custom deleters.

§ 5. Breaking changes

§ 5.1. Rejecting `0`, `{}` as arguments for the pointer-taking member functions

Currently the pointer taking member functions for single-object `default_delete` and `unique_ptr` have varying support for taking `0` or `{}` for their pointer arguments.

— `unique_ptr<int>(0)` and `unique_ptr<int>({})` are ill-formed (ambiguous for the `pointer` and `nullptr_t` taking members.

— `unique_ptr<int>(0, default_delete<int>{})` and `unique_ptr<int>({}, default_delete<int>{})` are well-formed.

— `uptr.reset(0)` and `uptr.reset({})` are well-formed.

This proposal makes no effort in preserving the current behavior here, and would possibly break usages depending on the well-formed constructs above.

§ 5.2. Rejecting class types as arguments for the pointer-taking member functions

Currently the pointer taking member functions for single-object `default_delete` and `unique_ptr` are non-templates, therefore they accept class types that convert to `pointer`.

This proposal replaces `default_delete::operator()` with a function template that takes `U*` with a deduced `U` type. This parameter can't take an argument that has class type and therefore break such usages. In this proposal the pointer-taking member functions of `unique_ptr` check whether the deleter can be directly called with the argument, therefore those member functions also don't take arguments of class type for specializations where the deleter is `default_delete`.

§ 6. Testing the changes on LLVM

Arthur O'Dwyer made an LLVM project fork where a similar constraint corresponding to the R0 revision of this paper is applied for the converting constructor of single-object `default_delete` in libc++. This fork was tested against compiling the LLVM project codebase. One of the libc++ tests failed to compile, it originally had undefined behavior (https://reviews.llvm.org/D90536). Later one similar bug found in production code in LLVM https://reviews.llvm.org/D154776

To my knowledge no false positives were found.

§ 7. Prior art

Boost.Move implements `unique_ptr` and `default_delete` that protects against missing virtual destructors. Its implementation has some notable differences to this proposal:

— It might produce ill-formed code for otherwise correct conversions to target types with destroying `operator delete`.

— It's not SFINAE-friendly.

— It only constrains the pointer-taking member functions of `unique_ptr` if the deleter is `default_delete`.

§ 8. Acknowledgements

I would like to thank Arthur O'Dwyer for his work to test the proposed changes on the LLVM codebase. I would like to thank Peter Sommerlad for discussing the proposal and evaluating whether a library implementation would be possible for *`has-destroying-delete`*.

§ 9. Wording

Wording is relative to [N4981].

§ 9.1. [unique.ptr.dltr.general]

…

3 The member functions of `default_delete` use the following exposition only concepts:

```
template <class T>
concept has-destroying-delete = see below;

template <class T, classU>
concept safely-convertible-for-delete = see below;
```

4 For types `T` and `U`, *`safely-convertible-for-delete`*`<T, U>` is:

(4.1) — `false` if either of `T` or `U` is not an object type, otherwise

(4.2) — `true` if `T(*)[]` is convertible to `U(*)[]`, otherwise

(4.3) — `false` if `reinterpret_cast<U*>((T*)nullptr)` is ill-formed (casts away const), otherwise

(4.4) — `false` if either of `T` or `U` is not a class type, otherwise

(4.5) — ill-formed outside of immediate context if both `T` and `U` are incomplete types, otherwise

(4.6) — `false` if `U` is an incomplete type, otherwise

(4.7) — `false` if `T` is an incomplete type and `U` is final, otherwise

(4.8) — `false` if `T` is an incomplete type and `has_virtual_destructor_v<U> || `*`has-destroying-delete`*`<U>` is `false`, otherwise

(4.9) — ill-formed outside of immediate context if `T` is an incomplete type, otherwise

(4.10) — `true` if `T*` is convertible to `U*` and `has_virtual_destructor_v<U> || `*`has-destroying-delete`*`<U>` is `true`, otherwise

(4.11) — `false`.

§ 9.2. [unique.ptr.dltr.dflt]

```
namespace std {
  template<class T> struct default_delete {
    constexpr default_delete() noexcept = default;
    template<class U> constexpr default_delete(const default_delete<U>&) noe
    constexpr void operator()(T*) const;
    template<class U> constexpr void operator()(U*) const;
  };
}
```

```
template constexpr default_delete(const default_delete<U>& other) noexcept;
```

1 *Constraints*: U* is implicitly convertible to T*. *safely-convertible-for-delete*<U, T> is true.

2 *Effects*: Constructs a `default_delete` object from another `default_delete<U>` object.

```
constexpr void operator()(T*) const;
template<class U> constexpr void operator()(U*) const;
```

3 *Constraints*: *safely-convertible-for-delete*<U, T> is true.

4 *Mandates*: `T` is a complete type.

5 *Effects*: Calls `delete` on `ptr`.

§ 9.3. [unique.ptr.single.general]

```
namespace std {
  template<class T, class D = default_delete<T>> class unique_ptr {
  public:
    using pointer      = see below;
    using element_type = T;
    using deleter_type = D;

    // [unique.ptr.single.ctor], constructors
    constexpr unique_ptr() noexcept;
    constexpr explicit unique_ptr(type_identity_t<pointer> p) noexcept;
    template<class U> explicit unique_ptr(U p) noexcept;
    constexpr unique_ptr(type_identity_t<pointer> p, see below d1) noexcept;
```

```
    constexpr unique_ptr(type_identity_t<pointer> p, see below d2) noexcept;
    template<class U> constexpr unique_ptr(U p, see below d1) noexcept;
    template<class U> constexpr unique_ptr(U p, see below d2) noexcept;
    constexpr unique_ptr(unique_ptr&& u) noexcept;
    constexpr unique_ptr(nullptr_t) noexcept;
    template<class U, class E>
      constexpr unique_ptr(unique_ptr<U, E>&& u) noexcept;

    // [unique.ptr.single.dtor], destructor
    constexpr ~unique_ptr();

    // [unique.ptr.single.asgn], assignment
    constexpr unique_ptr& operator=(unique_ptr&& u) noexcept;
    template<class U, class E>
      constexpr unique_ptr& operator=(unique_ptr<U, E>&& u) noexcept;
    constexpr unique_ptr& operator=(nullptr_t) noexcept;

    // [unique.ptr.single.observers], observers
    constexpr add_lvalue_reference_t<T> operator*() const noexcept(see below
    constexpr pointer operator->() const noexcept;
    constexpr pointer get() const noexcept;
    constexpr deleter_type& get_deleter() noexcept;
    constexpr const deleter_type& get_deleter() const noexcept;
    constexpr explicit operator bool() const noexcept;

    // [unique.ptr.single.modifiers], modifiers
    constexpr pointer release() noexcept;
    constexpr void reset(pointer p = pointer()) noexcept;
    constexpr void reset(nullptr_t = nullptr) noexcept;
    template<class U> constexpr void reset(U p) noexcept;
    constexpr void swap(unique_ptr& u) noexcept;

    // disable copy from lvalue
    unique_ptr(const unique_ptr&) = delete;
    unique_ptr& operator=(const unique_ptr&) = delete;
  };
```

…

§ 9.4. [unique.ptr.single.ctor]

…

~~constexpr explicit unique_ptr(type_identity_t<pointer> p) noexcept;~~
template<class U> constexpr explicit unique_ptr(U p) noexcept;

5 *Constraints*: `is_pointer_v<deleter_type>` is `false` ~~and~~ , `is_default_constructible_v<deleter_type>` is `true` , `get_deleter()(p)` is well-formed and `U` is convertible to `pointer` .

…

~~constexpr unique_ptr(type_identity_t<pointer> p, const D& d) noexcept;~~
~~constexpr unique_ptr(type_identity_t<pointer> p, remove_reference_t<D>&& d) noexcept;~~
template<class U> constexpr unique_ptr(U p, const D& d1) noexcept;
template<class U> constexpr unique_ptr(U p, remove_reference_t<D>&& d2) noexcept;

9 *Constraints*: ~~is_constructible_v<D, decltype(d)> is true~~.

(9.1) — `is_constructible_v<D, decltype(d)>` is `true`, and
(9.1.1) — `U` is `nullptr_t`, or
(9.1.2) — `get_deleter()(p)` is well-formed and `U` is convertible to `pointer`.

…

§ 9.5. [unique.ptr.single.modifiers]

…

~~constexpr void reset(pointer p = pointer()) noexcept;~~
constexpr void reset(nullptr_t p = nullptr) noexcept;

3 *Effects*: Equivalent to `reset(pointer())`.

template<class U> constexpr void reset(U p) noexcept;

4 *Constraints*: `get_deleter()(p)` is well-formed and `U` is convertible to `pointer`.

5 *Effects*: Assigns `p` to the stored pointer, and then, with the old value of the stored pointer, `old_p`, evaluates `if (old_p) get_deleter()(old_p);`

…

§ 9.6. [unique.ptr.runtime.ctor]

```
template<class U> constexpr explicit unique_ptr(U p) noexcept;
```

1 This constructor behaves the same as the constructor in the primary template that takes a single parameter of type ~~`pointer`~~ `U` .

2 *Constraints*:

(2.1) — `U` is the same type as `pointer`, or

(2.2) — `pointer` is the same type as `element_type*`, `U` is a pointer type `V*`, ~~and~~ `V(*)[]` is convertible to `element_type(*)[]` , and `get_deleter()(p)` is well-formed .

```
template<class U> constexpr unique_ptr(U p, see below d) noexcept;
template<class U> constexpr unique_ptr(U p, see below d) noexcept;
```

3 These constructors behave the same as the constructors in the primary template that take a parameter of type ~~`pointer`~~ `U` and a second parameter.

4 *Constraints*:

(4.1) — `U` is the same type as `pointer`,

(4.2) — `U` is `nullptr_t`, or

(4.3) — `pointer` is the same type as `element_type*`, `U` is a pointer type `V*`, ~~and~~ `V(*)[]` is convertible to `element_type(*)[]` , and `get_deleter()(p)` is well-formed .

…

§ 9.7. [unique.ptr.runtime.modifiers]

…

```
template<class U> constexpr void reset(U p) noexcept;
```

2 This function behaves the same as the `reset` member of the primary template.

3 *Constraints*:

(3.1) — `U` is the same type as `pointer`, or

(3.2) — `pointer` is the same type as `element_type*`, `U` is a pointer type `V*`, ~~and~~ `V(*)[]` is convertible to `element_type(*)[]` , and `get_deleter()(p)` is well-formed .

§ Appendix A

§ Possible implementation of *safely-convertible-for-delete*

```
template <typename From, typename To>
concept casts-away-const // exposition-only
= not requires (From* ptr) {
  reinterpret_cast<To*>(ptr);
};

template <typename From, typename To>
concept safely-convertible-classes-for-delete // exposition-only
= []{
  static_assert(
    requires { sizeof(From); };
    || requires {
        sizeof(To);
        requires is_final_v<To>
                 || !(has_virtual_destructor_v<To> || has-destroying-delete
    }
  );
  return requires (From* ptr, void(*fn)(To*)) {
    fn(ptr);
    requires has_virtual_destructor_v<To> || has-destroying-delete<To>;
  }
}();

template <typename From, typename To>
concept safely-convertible-for-delete // exposition-only
= is_object_v<From> && is_object_v<To>
  && (
```

```
    is_convertible_v<From(*)[], To(*)[]>
    || (
      not casts-away-const<From, To>
      && is_class_v<From>
      && is_class_v<To>
      && safely-convertible-classes-for-delete<From, To>
    )
  );
```

§ Destroying `operator delete`

C++20 introduced destroying `operator delete`. Because of this `delete ptr` might be defined even if the static type of the pointed object does not have a virtual destructor and the static type does not match the dynamic type of the pointed object. Consider the following program:

```
#include <memory>
#include <new>

struct Base {
    void operator delete(Base* ptr, std::destroying_delete_t);
};

struct Derived : Base {};

void Base::operator delete(Base* ptr, std::destroying_delete_t) {
    ::delete static_cast<Derived*>(ptr);
}

int main() {
    std::unique_ptr<Base> base_ptr = std::make_unique<Derived>();
}
```

This program is well-formed in the current draft and has defined behavior. [P0722R1] provides a motivating example with a similar class hierarchy (section "Dynamic dispatch without vptrs").

The R0 version of this proposal rejected the conversion above and suggested to add a customization point for users of destroying operator delete to optionally enable the conversion.

The consensus of the mailing list review was that such a customization point is unnecessary and breaking correct code involving destroying operator delete by default is undesirable.

The constraints added in the current proposal allow the conversion if the target type has destroying operator delete.

§ References

§ Informative References

[N4981]

Thomas Köppe. *Working Draft, Standard for Programming Language C++*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/n4981.pdf

[P0722R1]

Andrew Hunter; Richard Smith. *Efficient sized delete for variable sized classes*. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0722r1.html